



Retinal Review Workbench

Developer Technical Guide

Architecture, implementation, and change guide for the current main branch

Software version	0.7.0
Guide version	1.0 (2026-09-23)
Document ID	RRW-DEV-TG-001
Audience	Developers, technical lead, hospital IT
Verified against	main at ec74af8 (code, tests, and scripts traced)
Companions	DEVELOPER_HANDOFF.md, FEATURE_IMPLEMENTATION_MAP.md, WHERE_TO_CHANGE_WHAT.md, docs/adr/

Research and clinical-support proof of concept for public or synthetic data. Model output is assistive evidence; human review remains authoritative.

Table of contents

1	How to Use This Guide	3
2	System Context and Containers	4
2.1	Goals and constraints	4
2.2	System Context (C4 level 1)	4
2.3	Containers (C4 level 2)	5
3	Image Source and Sampling Boundary	6
3.1	Hospital storage to Workbench admission	6
3.2	Source class versus Workbench grade	7
3.3	What the software contributes to coverage monitoring	7
4	Repository Map and Technology Stack	8
4.1	Repository map	8
4.2	Verified technology stack	9
5	Runtime Profiles and Startup	10
5.1	Profiles	10
5.2	Workstation startup (START.cmd)	10
5.3	Model-server setup and startup (GPU host)	10
6	Image Admission and Patient/Eye Resolution	11
6.1	Admission (dr_support/services/admission.py)	12
6.2	Resolver (dr_support/services/resolver.py)	12
6.3	Confirm Image (POST /v1/cases/{id}/confirm-image)	13
7	Remote Model Inference	13
7.1	Request path on the workstation	13
7.2	Source SHA versus analysis SHA	14
7.3	Model API (GPU host)	14
8	Review State, Grades, and Annotations	15
8.1	Case record and concurrency	15
8.2	Confirm DR Grade (POST /v1/cases/{id}/review)	15
8.3	Human annotations (PUT /v1/cases/{id}/annotations)	15
8.4	AI ROI correction and removal (POST /v1/cases/{id}/lesion-review)	15
8.5	Confirm Annotation	16
8.6	UI conveniences (no clinical state)	16
9	Dataset Readiness and Export	16
9.1	Task-specific readiness	16
9.2	Export package	17
10	API and Contract Map	17
10.1	Review workstation API	17
10.2	Model API (GPU host)	18
11	Testing and Validation	19

- 11.1 Commands 19
 - 11.2 What the suites cover 19
 - 11.3 Known environment prerequisites 19
- 12 Deployment, Security, and Privacy 20**
 - 12.1 Topology 20
 - 12.2 Security and privacy controls in code 20
- 13 Documentation as Code 21**
- 14 Limitations and Glossary 22**
 - 14.1 Known limitations 22
 - 14.2 Glossary 22

1 How to Use This Guide

This guide explains how Retinal Review Workbench is built on the current main branch and where to start when you change it. Every statement was traced to code, tests, or scripts. If code and this guide disagree, the code is authoritative; fix the guide.

The guide is **explanation and reference** in Diátaxis terms. Task-oriented material lives elsewhere, so it is not duplicated here:

Table 1.1: Where each kind of information lives.

You want to...	Go to
install, start, back up, or troubleshoot a workstation or GPU host	<i>Deployment and Operations Manual</i> (docs/pdfs/DEPLOYMENT_OPERATIONS_MANUAL.pdf) and docs/operations/INSTALLATION.md, docs/operations/MODEL_SERVER.md, docs/operations/CONFIGURATION.md
review cases as a clinician	<i>Clinician User Manual</i> (docs/pdfs/CLINICIAN_USER_MANUAL.pdf)
know how images are chosen and how coverage is monitored	<i>Team Image Sampling Requirements</i> (docs/pdfs/IMAGE_SAMPLING_REQUIREMENTS.pdf)
find the code for a feature	docs/developer/FEATURE_IMPLEMENTATION_MAP.md
make a specific kind of change safely	docs/developer/WHERE_TO_CHANGE_WHAT.md
take over ownership of the project	docs/developer/DEVELOPER_HANDOFF.md
understand why something is designed this way	docs/adr/ (architecture decision records)
check the export schema	docs/reference/DATASET_MANIFEST.md
check UI tokens and interaction rules	DESIGN.md

The chapter order follows a simplified arc42 outline: context and scope (Section 2, Section 3), building blocks (Section 4), runtime views (Section 5 to Section 10), then testing, deployment, and cross-cutting concerns (Section 11 to Section 14). Architecture diagrams use C4 levels (System Context, Container, Dynamic, Deployment). They are written in Mermaid under docs/adr/architecture/ and rendered with python scripts/docs/build_docs.py diagrams.

Important – rules that bind every change

AGENTS.md governs the workflow: bounded feat/, fix/, or docs/ branches, never direct implementation on main, and a full validation run before merge. It also lists the protected areas. Admission semantics, patient/eye resolver semantics, model/provider contracts, annotation provenance, original-image coordinates, and clinician-review semantics must not change without an owner-approved normative specification.

2 System Context and Containers

2.1 Goals and constraints

- **Goal.** Turn a clinician-selected retinal image into a reviewed, provenance-preserving dataset record. Optional model evidence may support the review, but it never replaces the clinician's decision.
- **Scope.** A research and clinical-support proof of concept for public/synthetic data and approved hospital use. It is not autonomous diagnosis, referral, or a regulated medical device.
- **Hard constraints** (enforced in code):
 - the review workstation never loads model weights (APP_PROFILE=review requires MODEL_RUNTIME=remote);
 - original admitted images are immutable;
 - human review is authoritative;
 - secrets stay server-side.

2.2 System Context (C4 level 1)

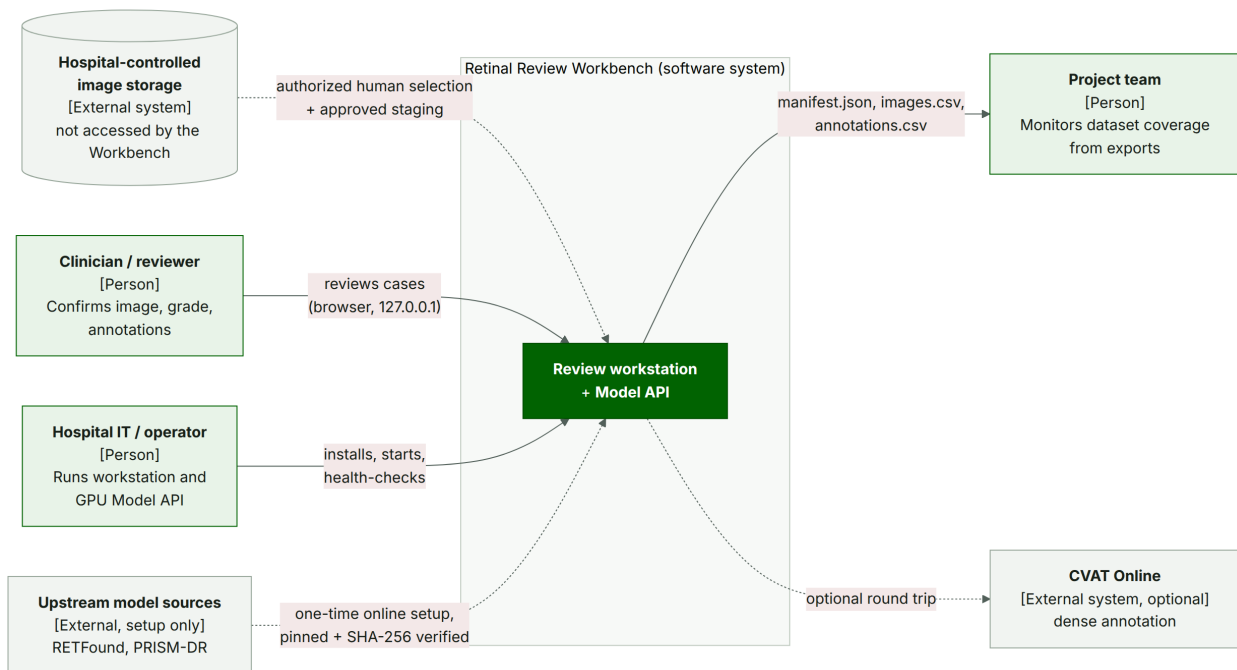


Figure 2.1: System Context. Source: docs/adr/architecture/01-system-context.mmd.

Table 2.1: Context relationships.

Actor or system	Relationship
Clinician / reviewer	Uses the browser UI on the workstation (127.0.0.1:8000/app/). Owns every confirmation.
Project team	Receives dataset exports and monitors coverage outside the software (Section 3).
Hospital IT / operator	Installs and runs the workstation and the GPU Model API.
Hospital-controlled image storage	Not accessed by the Workbench. Images arrive only through authorized human selection and approved staging.
CVAT Online	Optional dense-annotation round trip (dr_support/cvat.py, dr_support/sync.py).

Actor or system	Relationship
Upstream model sources	Downloaded once during online setup, then pinned and SHA-256 verified.

2.3 Containers (C4 level 2)

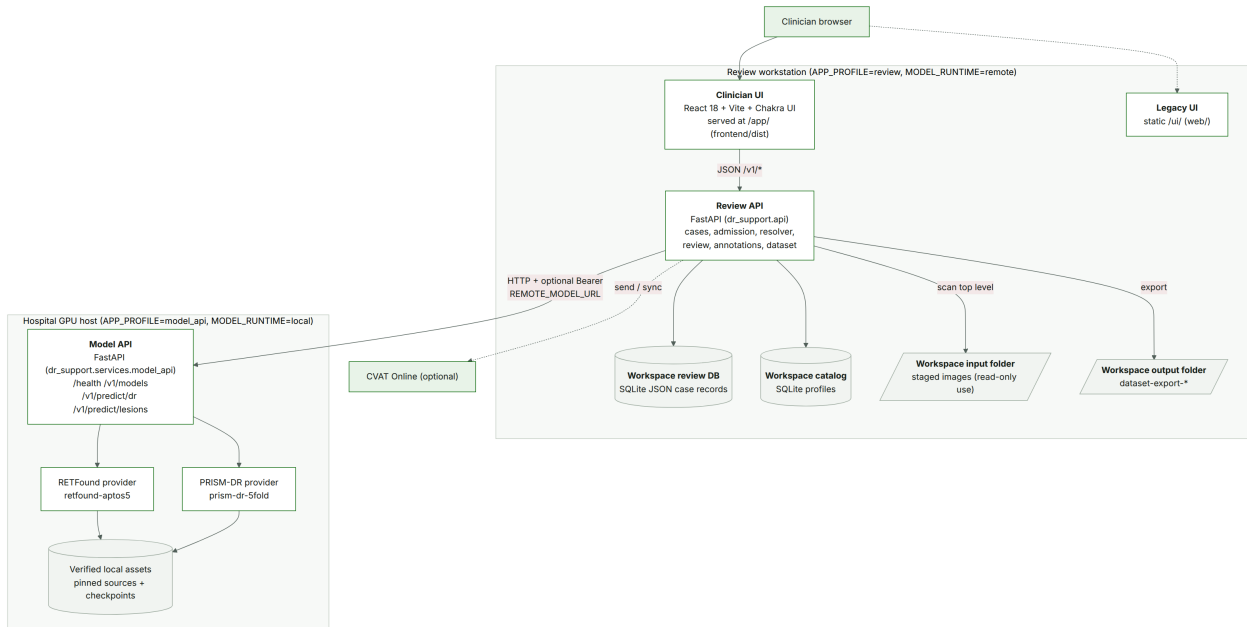


Figure 2.2: Container view. Source: docs/adr/architecture/02-container.mmd.

Table 2.2: Containers.

Container	Technology	Code
Clinician UI	React 18 + TypeScript (strict) + Vite 5 + Chakra UI 2, served from frontend/dist at /app/	frontend/src/**
Legacy UI	Static HTML/JS at /ui/ (compatibility only)	web/**
Review API	FastAPI app built by <code>dr_support.api._factory.create_app</code>	<code>dr_support/api/**</code> , <code>dr_support/workflow.py</code> , <code>dr_support/services/**</code>
Workspace review DB	SQLite, one JSON document per case (<code>cases(id, data)</code>), optimistic revision	<code>dr_support/store.py</code>
Workspace catalog	SQLite list of Workspace profiles (input, output, DB path)	<code>dr_support/services/workspaces.py</code>
Workspace input/output folders	File system; input is read-only in practice; output receives <code>dataset-export-*</code>	<code>services/admission.py</code> , <code>services/dataset.py</code>
Model API	FastAPI app from <code>dr_support.services.model_api.create_app</code> on the GPU host	<code>dr_support/services/model_api.py</code> , <code>dr_support/providers/**</code>

One code base serves every container. `dr_support/app.py` selects the surface with `APP_PROFILE` (Section 5).

3 Image Source and Sampling Boundary

3.1 Hospital storage to Workbench admission

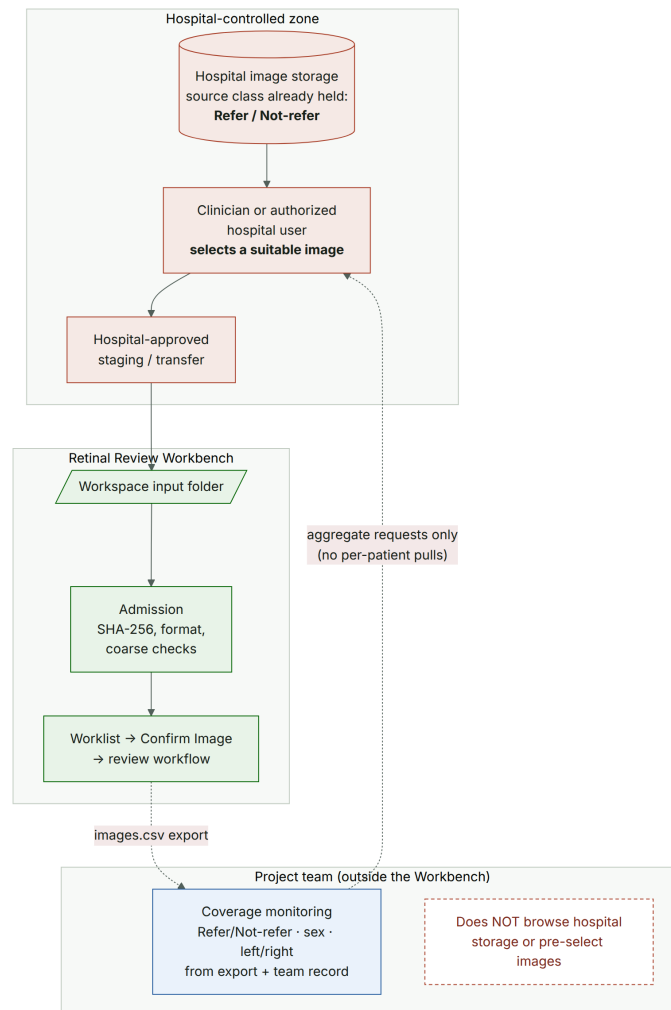


Figure 3.1: Image ownership and staging boundary. Source: docs/adr/architecture/03-image-staging-boundary.mmd.

1. **Hospital-controlled storage** holds candidate images. It is outside the software.
2. **Authorized human selection.** A clinician or authorized hospital user chooses each image. The project team and the software never browse storage or pre-select images (owner decision, IMG-SAMP-001).
3. **Approved staging.** The hospital's own process copies the original file into the Workspace input folder.
4. **Workbench admission.** scan_input_folder reads **only that folder, top level only**. It skips symlinks and ancillary files (.csv .json .md .txt .log .toml .xml .yaml .yml), records SHA-256 identity, and applies conservative admission rules (Section 6).

There is no PACS query, storage crawler, or automatic patient selection anywhere in the code. Adding one would cross an owner-decided boundary and needs a new normative specification.

3.2 Source class versus Workbench grade

Safety – two taxonomies, never merged

The source hospital's **Refer / Not-refer** class is **not** stored anywhere in the Workbench. The Workbench records a **clinician DR grade 0–4** (reviewed_grade, with grade_review_source = AI_ACCEPTED / AI_CORRECTED / MANUAL) and **annotations**. These are different taxonomies, produced at different times by different processes. Do not add code that derives one from the other.

The team links the source class (and sex, where approved) to exported images outside the software. It joins its own record to `images.csv` by `image_id` or `image_sha256` (IMG-SAMP-013, IMG-SAMP-091). If the owner later wants the source class inside the Workbench, that is a contract change: it touches admission, the case schema, `DATASET_MANIFEST.md`, and the tests, and it needs a spec first.

3.3 What the software contributes to coverage monitoring

Table 3.1: Coverage dimensions and where they come from.

Coverage dimension	Held by the Workbench?	Export column
Refer / Not-refer	No	None; the team record holds it
Sex	No	None; the team record holds it
Laterality	Yes: clinician-confirmed in Confirm Image	laterality, laterality_resolution_method (MANUAL = confirmed)
Patient grouping	Yes: pseudonymous key	patient_key, training_group_key
Exact duplicates	Yes: SHA-256 aliases	integrity status in the case record (DUPLICATE_CONTENT)
Suitability	Yes: admission state	modality_admission, quality_state, queue_state

The full sampling policy, including planning targets and open owner decisions, is in the *Team Image Sampling Requirements*. Its register (`docs/clinician/sampling/REQUIREMENTS_TRACEABILITY.md`) marks which items are software-enforced.

4 Repository Map and Technology Stack

4.1 Repository map

Table 4.1: Repository map.

Path	Contents
dr_support/app.py	Profile-aware create_app; enforces APP_PROFILE × MODEL_RUNTIME invariants.
dr_support/run.py	Console entry (python -m dr_support.run, dr-support-run); reads HOST, PORT, WORKERS.
dr_support/api/	Review API factory (_factory.py), Workspace routes (workspaces.py), dataset routes (dataset.py).
dr_support/workflow.py	Case, review, Confirm Image, resolver, admission-override, queue, annotation, AI-ROI, CVAT, and manual-sync routes.
dr_support/services/	admission.py (scan and admission rules), resolver.py (patient/eye), dataset.py (readiness and export), workspaces.py (catalog and profiles), pickers.py (native folder/DB pickers), model_gateway.py (connection probe), model_api.py (GPU Model API).
dr_support/imaging/	Integrity contracts, DICOM ingestion (dicom.py), handler registry, deterministic display/analysis derivatives (derivatives.py).
dr_support/providers/	retfound.py, prism.py (+ prism_assets.json), remote.py (HTTP client providers), mock.py (synthetic fixtures only), assets.py.
dr_support/contracts/	Pydantic contracts: admission, resolver, workspace, model gateway, inference results.
dr_support/presentation.py	Bounded lesion-review view, detection IDs, review evidence, annotation-set hash.
dr_support/store.py	SQLite case store with additive JSON migration defaults.
dr_support/cvat.py, sync.py	Optional CVAT Online transport and round-trip sync.
dr_support/setup_models.py, model_assets.py	One-time asset acquisition (--model all) and read-only verification (--verify).
frontend/src/	React UI: pages/, components/{shell,worklist,review,workspace,settings,common}, lib/ (API client, preferences, navigation), theme/.
frontend/tests/	Vitest + Testing Library tests (jsdom).
tests/	Pytest suite plus root Node smoke tests (*.test.cjs).
web/	Legacy static UI (/ui/).
scripts/model-server/	setup.sh, start.sh, healthcheck.sh, install-service.sh for the GPU host.
scripts/docs/	build_docs.py (every document), check_docs.py (documentation QA), generate_manifest_example.py.
deployment/	systemd unit and model-server.env.example.
docs/	Manuals (Quarto sources), specs, runbooks, architecture sources, ADRs, demo decks.
local-state/	Runtime state only (gitignored). Never source data.

4.2 Verified technology stack

Versions are taken from `pyproject.toml` (exact pins) and `frontend/package-lock.json` (resolved versions).

Table 4.2: Verified stack.

Layer	Component	Version / constraint
Runtime	Python	$\geq 3.11, < 3.13$ (3.12 recommended by <code>START.cmd</code>)
Backend	FastAPI / Uvicorn / Pydantic / httpx / Pillow	0.141.1 / 0.53.0 / 2.13.5 / 0.28.1 / 12.3.0
DICOM (extra dicom)	pydicom, pylibjpeg, -libjpeg, -openjpeg	3.0.1, 2.1.0, 2.3.0, 2.5.0
Models (extra models, GPU host)	torch / torchvision / timm / ultralytics / sahi / ensemble-boxes / gdown	2.5.1 / 0.20.1 / 0.9.2 / 8.4.8 / 0.12.6 / 1.0.9 / 6.4.0
Tests (extra test)	pytest / ruff	8.4.2 / 0.13.0 (Ruff rules E9, F; line length 110)
Browser tests (extra)	playwright	1.57.0
Frontend	React / React DOM / React Router	18.3.1 / 18.3.1 / 6.30.6
Frontend	Chakra UI / Emotion / framer-motion / lucide-react	2.10.10 / 11.x / 11.18.2 / 0.460.0
Frontend build	Vite / @vitejs/plugin-react / TypeScript	5.4.21 / 4.7.0 / 5.9.3 (strict)
Frontend tests	Vitest / jsdom / Testing Library React	2.1.9 / 25.0.1 / 16.3.3
Root smoke tests	Node.js + jsdom	jsdom 26.1.0
Model sources	RETFound revision / PRISM-DR revision	ae9a9ecf... / 79637440... (full SHAs in <code>providers/*.py</code>)
Documentation	Quarto + XeLaTeX; Mermaid CLI; Playwright (guide PDF)	See Section 13

Frontend build facts (`frontend/vite.config.ts`): `base: '/app/'`, `output frontend/dist` (gitignored), `sourcemap: true`, `target: es2020`, and `alias @ → src`. In development, the Vite server on 5173 proxies `/v1`, `/ui`, and `/health` to `127.0.0.1:8000` with `changeOrigin: false`, which keeps the same-origin guard meaningful.

5 Runtime Profiles and Startup

5.1 Profiles

`dr_support/app.py` builds one of three applications. Misconfiguration raises at startup; it never falls back silently.

Table 5.1: Runtime profiles.

APP_PROFILE	Required MODEL_RUNTIME	Surface	Used by
review (default)	remote	Review API + <code>/app/</code> + <code>/ui/</code> ; no sample or demo fixtures; model calls go to <code>REMOTE_MODEL_URL</code>	Clinician workstation (<code>START.cmd</code>)
model_api	local	<code>/health</code> , <code>/v1/models</code> , <code>/v1/predict/dr</code> , <code>/v1/predict/lesions</code> ; CUDA enforced unless <code>DR_SUPPORT_RELAX_DEVICE=1</code>	GPU host (<code>start.sh</code>)
full	local	Both surfaces merged, with synthetic fixtures and public samples; CPU allowed	Local demos and development only

`dr_support.api:app` is a separate legacy entry point used by the root UI smoke test (`tests/ui.test.cjs`). It does **not** apply the profile invariants and it includes the synthetic fixture and the public samples.

5.2 Workstation startup (`START.cmd`)

1. Create `.venv` with Python 3.12 (Windows py launcher, or `uv` as fallback) and install `-e ..`
2. If `frontend/dist/index.html` is missing: `npm ci` (first run only), then `npm run build` in `frontend/`.
3. Set `APP_PROFILE=review` and `MODEL_RUNTIME=remote`, then run `python -m dr_support.run`, which serves `127.0.0.1:8000` by default.
4. At startup, `WorkspaceManager` opens the catalog (`DR_SUPPORT_WORKSPACE_CATALOG`, default `local-state/bridge/workspaces.sqlite`). It then opens the active `Workspace`'s review DB, or the fallback DB (`DR_SUPPORT_STATE`, default `local-state/bridge/reviews.sqlite`), and scans the active input folder once.

`REMOTE_MODEL_URL` and the optional `REMOTE_MODEL_TOKEN` come from the process environment.

The connection can also be tested and saved at runtime through **Settings** → **Model connection** (`/v1/model-connection*`). The token is held in process memory only and is never returned by the API.

5.3 Model-server setup and startup (GPU host)

Scripts are in `scripts/model-server/`.

Table 5.2: Model server lifecycle.

Step	Script	Network	Result
One-time setup	setup.sh	Online	Creates .venv, installs [models, dicom], runs <code>python -m dr_support.setup_models --model all --smoke</code> , clones pinned sources, downloads checkpoints, verifies hashes, runs synthetic smoke inference, prints READY.
Normal start	start.sh	None needed	Sets <code>APP_PROFILE=model_api</code> , <code>MODEL_RUNTIME=local</code> , <code>INFERENCE_DEVICE=cuda:0</code> , <code>HOST=0.0.0.0</code> , <code>PORT=7860</code> , <code>WORKERS=1</code> , <code>MODEL_REQUIRE_VERIFIED_ASSETS=1</code> , runs <code>setup_models --verify (read-only)</code> , then <code>dr_support.run</code> . Missing or modified assets fail closed.
Health	healthcheck.sh	LAN	Calls <code>/health</code> and <code>/v1/models</code> ; fails unless health reports PASS.
Service	<code>sudo ./install-service.sh</code>	None	Installs deployment/ <code>dr-support-model-api.service</code> (user <code>drsupport</code> , <code>/opt/dr-support-screening-poc</code> , <code>/etc/dr-support/model-server.env</code>).

Pinned identities: RETFound revision and WEIGHT_SHA256 in `dr_support/providers/retfound.py`; PRISM-DR revision in `providers/prism.py`; the PRISM archive and per-fold weight hashes in `providers/prism_assets.json`. Changing any of them is a model-provenance change (see `AGENTS.md`).

6 Image Admission and Patient/Eye Resolution

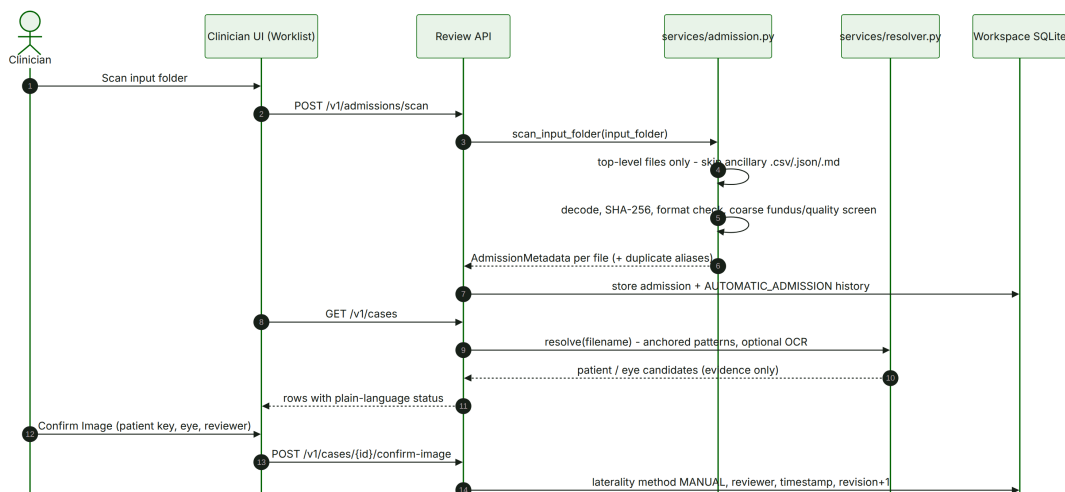


Figure 6.1: Admission and Confirm Image, dynamic view. Source: `docs/adr/architecture/04-image-admission-flow.mmd`.

6.1 Admission (dr_support/services/admission.py)

Trigger. POST /v1/admissions/scan (Worklist **Scan input folder**), plus one scan at startup when a Workspace is active. apply_scan in api/_factory.py merges the result. It replaces automatic records, **preserves manual admission decisions** (only identity and integrity fields are refreshed), and marks references that disappeared as SOURCE_MISSING without deleting the case.

Per-file decision (inspect_file):

1. The extension must be one of .jpg .jpeg .png .tif .tiff .dcm .dicom. Anything else becomes UNSUPPORTED_FORMAT (REJECTED_INVALID).
2. Decode and check that the format matches the extension (FORMAT_EXTENSION_MISMATCH otherwise). Compute the **source SHA-256**. For byte-identical files, the image ID is content-derived, so duplicates collapse into one case with several aliases (DUPLICATE_CONTENT).
3. Raster images go through a coarse screen (_classify_decoded):
 - **Quality flags:** VERY_SMALL_IMAGE (under 128 px), EXTREME_ASPECT_RATIO (outside 0.45–2.4), NEAR_UNIFORM_IMAGE, EXTREME_BRIGHTNESS.
 - **Fundus signals:** geometry, colour, texture, and field. All four are required for automatic FUNDUS_ACCEPTED (FUNDUS_PLAUSIBLE). Otherwise the image is NEEDS_REVIEW (FUNDUS_UNCERTAIN).
 - Readable images are never auto-rejected for being uncertain.
4. DICOM goes through imaging/dicom.py:
 - Single-frame ophthalmic photography with a supported transfer syntax → NEEDS_REVIEW (DICOM_REVIEW_REQUIRED), ready for **Confirm Image**.
 - Recognised non-fundus modality → DICOM_UNSUPPORTED_MODALITY, shown as **Unsupported modality**.
 - Codec, multi-frame, or decode problems keep their specific status.
 - ImageLaterality / Laterality are read as a metadata candidate only.

States (contracts/admission.py):

- modality_admission is one of {FUNDUS_ACCEPTED, NEEDS_REVIEW, REJECTED_NON_FUNDUS, REJECTED_INVALID}
- quality_state is one of {GRADABLE, UNGRADABLE, NEEDS_REVIEW, NOT_EVALUATED}
- is_inference_eligible requires FUNDUS_ACCEPTED and quality GRADABLE or NOT_EVALUATED.
- clinician_view maps these states to the plain-language labels in the UI.

Manual overrides. POST /v1/cases/{id}/admission accepts ACCEPT_RETINAL, MARK_NON_FUNDUS, QUALITY_ACCEPTABLE, QUALITY_INADEQUATE, and LEAVE_UNRESOLVED. It records ADMISSION_OVERRIDE history with the previous and new states.

6.2 Resolver (dr_support/services/resolver.py)

- **Filename-first.** The resolver matches anchored patterns, for example PAT0001_L or PAT0001-RIGHT-2. A match yields a patient key and laterality with method FILENAME. Ambiguous names yield NEEDS_CONFIRMATION.
- **Optional OCR.** An injectable OCRAdapter returns candidates only. The default is DisabledOCRAdapter, so OCR is off. It is consulted only when the filename lacks a patient key or an eye. Conflicts between filename and OCR become CONFLICT and are never auto-resolved.
- **When it runs.** Resolution happens once per case (ensure_resolution in workflow.py). A later manual decision is never overwritten.
- **Patient keys.** Keys are normalised to the pseudonymous pattern `^[A-Z][A-Z0-9_-]{2,31}$` with at least one digit.

6.3 Confirm Image (POST /v1/cases/{id}/confirm-image)

This is the first confirmation milestone. It is one request containing revision, reviewer, optional patient_key, laterality is one of {LEFT, RIGHT, UNKNOWN}, and a note. The endpoint:

- sets admission NEEDS_REVIEW → FUNDUS_ACCEPTED (MANUAL_CONFIRM_IMAGE), and quality NEEDS_REVIEW → GRADABLE;
- records reviewed_by and reviewed_at;
- sets the patient key (or UNLINKED) and the laterality with method MANUAL;
- appends history and increments revision.

The export treats the confirmed value as authoritative (laterality_resolution_method = MANUAL).

7 Remote Model Inference

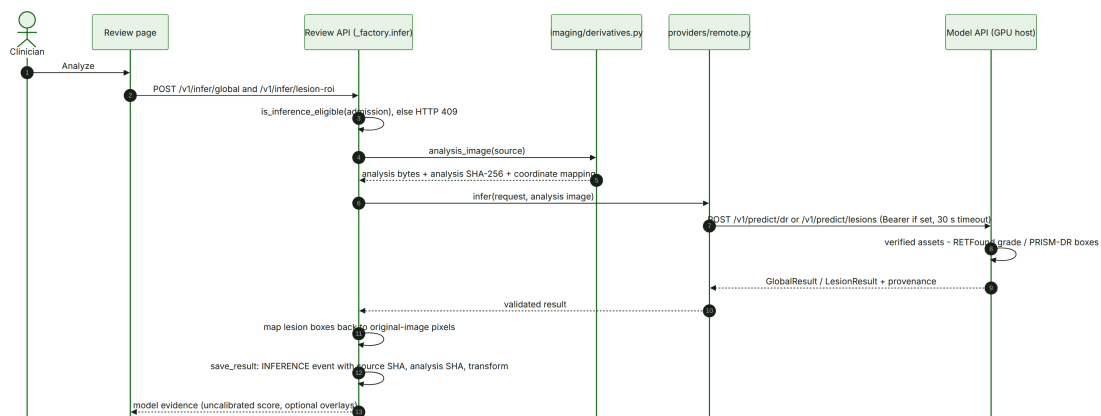


Figure 7.1: Optional remote inference, dynamic view. Source: docs/adr/architecture/05-remote-inference-flow.mmd.

7.1 Request path on the workstation

POST /v1/infer/global and POST /v1/infer/lesion-roi both call infer() in dr_support/api/_factory.py:

1. The image must be admitted and is_inference_eligible. Otherwise the API returns 409 Image needs review before analysis.
2. mock-* model IDs are allowed only on SYNTHETIC images.
3. DerivativeService.analysis_image(source) prepares a deterministic **analysis representation**:
 - analysis-source-v1 for JPEG and PNG, which pass through unchanged;
 - analysis-tiff-png-rgb-v1 for TIFF;
 - analysis-dicom-png-rgb-v1 for DICOM.

The analysis image has its own SHA-256 and a CoordinateMapping back to the original pixels. A payload whose base64 encoding exceeds 20 MB returns 413; it is never downscaled silently.

4. A process-wide RLock serialises provider calls.
5. RemoteGlobalProvider / RemoteLesionProvider (providers/remote.py) POST the base64 analysis image and metadata to the Model API. Details:
 - optional Authorization: Bearer <REMOTE_MODEL_TOKEN>;
 - 30 s inference timeout and 5 s metadata timeout;
 - no redirects;
 - responses are validated against the result schemas.

6. Lesion boxes are mapped back to original-image pixels (`map_lesion_result_to_review`).
7. `save_result` stores the raw result and the `analysis_derivative` audit record, and appends an INFERENCE event with `source_sha256`, `analysis_sha256`, `transform_id`, `runtime`, and `latency`. A new **global** result resets the grade decision to PENDING so that a stale human grade never sits beside a new suggestion.

Errors are mapped to plain HTTP codes: 503 (remote not configured), 504 (timeout), 502 (remote error), 409 / 413 (derivative problems).

7.2 Source SHA versus analysis SHA

Table 7.1: Image identities.

Identity	What it hashes	Where it appears
Source SHA-256	The admitted original bytes. This is the case identity and never changes.	Admission record, <code>images.csv</code> , <code>image_sha256</code> , display headers, X-Source-SHA256, Models & Audit
Analysis SHA-256	The exact bytes sent to the model (possibly converted).	<code>analysis_derivative</code> , INFERENCE events, Models & Audit
Display derivative SHA	The browser-safe representation served by <code>/v1/images/{id}/display</code> .	ETag and X-Derivative-SHA256 headers

For JPEG and PNG, the analysis bytes equal the source bytes, so both hashes match. TIFF and DICOM produce a different analysis SHA. The mapping keeps every coordinate in original-image pixel space.

7.3 Model API (GPU host)

`dr_support/services/model_api.py` validates the Bearer token with a constant-time comparison when `REMOTE_MODEL_TOKEN` is set, checks the runtime and admission fields, and runs the local providers under a lock.

- **RETFound** (`providers/retfound.py`, model ID `retfound-aptos5`)
 - Pinned source revision and checkpoint SHA-256.
 - Returns a five-class grade suggestion with a softmax score. The score is **uncalibrated**.
- **PRISM-DR** (`providers/prism.py`, model ID `prism-dr-5fold`)
 - ROI cropper, then green-channel median and CLAHE preprocessing.
 - MA/HE/EX run on 1280 px tiles with 0.25 overlap; SE runs on the full image.
 - Five-fold ensemble; boxes are returned in original pixels.
 - The raw output is kept intact. `presentation.py` derives a bounded display subset (`REVIEW_MAX_PER_CLASS`, default 25; `REVIEW_MAX_TOTAL`, default 80) with stable `ai-<20 hex>` detection IDs.

Empty detections never mean “no lesion”. The UI and documentation must keep saying so.

8 Review State, Grades, and Annotations

8.1 Case record and concurrency

Each case is one JSON document in the Workspace SQLite store (`dr_support/store.py`). New fields are added as JSON defaults (additive migration), so there are no table rewrites. Every mutating request carries the client's revision. A mismatch returns 409 Case changed; reload. Success increments revision and appends an event. A same-origin middleware rejects cross-origin non-GET requests.

8.2 Confirm DR Grade (POST /v1/cases/{id}/review)

The Review contract is revision, action, reviewer, grade (0–4, strict integer), and comment.

Table 8.1: Review actions.

Action	Effect
ACCEPT	Final grade = model grade; grade_review_source = AI_ACCEPTED; state REVIEWED.
CORRECT_GRADE	Final grade = requested grade. The source is AI_CORRECTED when a model grade exists, otherwise MANUAL.
MARK_INCORRECT / ESCALATE	Clear the grade; state NEEDS_CORRECTION / ESCALATED.
CONFIRM_ANNOTATIONS	Annotation milestone (below). Leaves the DR-grade review record untouched.

The UI sends ACCEPT when the chosen grade equals the suggestion and CORRECT_GRADE otherwise. The clinician sees one button: **Confirm DR Grade**. Events carry `identity_assurance: LOCAL_POC_SELF_DECLARED`, because the reviewer name is not authentication.

8.3 Human annotations (PUT /v1/cases/{id}/annotations)

The request replaces the whole human set (maximum 500 shapes). Shapes are rectangle, polygon, point, or circle, with labels MICROANEURYSM, HEMORRHAGE, HARD_EXUDATE, or SOFT_EXUDATE. `clean_geometry` validates finite numbers inside the original image bounds. The saved event records added, removed, and changed shape IDs.

In the UI, AnnotationEditorPage autosaves 900 ms after the last edit when a reviewer is set. It shows the draft status and guards navigation (`window.confirm`) and page unload (`beforeunload`) while there are unsaved, saving, or failed changes.

8.4 AI ROI correction and removal (POST /v1/cases/{id}/lesion-review)

The LesionReviewAction contract has `detection_id` (`ai-[a-f0-9]{20}`), `action` (CONFIRM / REJECT / CORRECT), and an optional corrected label or rectangle. The UI (LesionActionPopover) exposes class correction and **Remove this detection** (REJECT). Each decision is appended to `ai_annotation_reviews` with the **original** label, rectangle, score, and model identity. Raw model output is never modified. `lesionPresentation.ts` shows a corrected ROI as “class – clinician corrected” **without** a score.

8.5 Confirm Annotation

`annotation_set_hash(case)` (`presentation.py`) hashes the active set: human shapes, AI decisions, and CVAT-imported shapes. `CONFIRM_ANNOTATIONS` stores that hash with reviewer and time, and sets `lesion_review_state = REVIEWED`. Any later annotation save, AI-ROI decision, or import changes the hash and moves the case back to `REQUIRES_CONFIRMATION` or `IMPORTED_REQUIRES_REVIEW`. Confirming an empty set is allowed. It means “reviewed”, not “no lesions”.

8.6 UI conveniences (no clinical state)

Table 8.2: Conveniences.

Feature	Code	Rule
Reviewer default	<code>frontend/src/lib/reviewerPreference.ts</code> (localStorage key <code>dr-support-screening.default-reviewer-history</code>).	Prefills empty or unattributed forms only; never rewrites history.
Next-action hints	<code>components/common/NextActionHint.tsx</code> , <code>lib/hintPreference.ts</code>	Guidance only.
Previous / Next	<code>lib/caseNavigation.ts</code> , <code>components/common/CaseNavigation.tsx</code>	Filename order, excluding excluded cases. Never saves or confirms.
Save & Next / Confirm DR Grade & Next	<code>AnnotationEditorPage.tsx</code> , <code>ClinicianReviewPage.tsx</code>	Performs the named save or confirm first, then navigates.

9 Dataset Readiness and Export

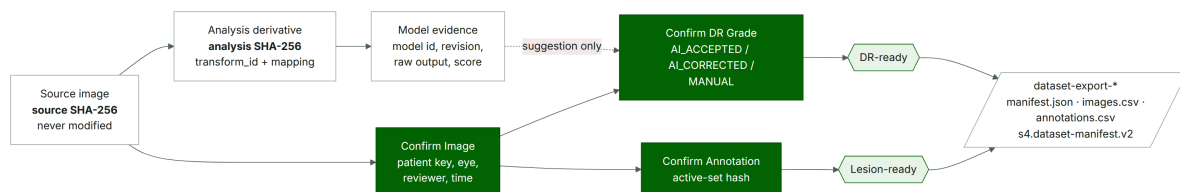


Figure 9.1: Dataset readiness and provenance. Source: `docs/adr/architecture/06-dataset-provenance-flow.mmd`.

`DatasetManifestService` (`dr_support/services/dataset.py`) serves GET `/v1/dataset/manifest` (preview) and POST `/v1/dataset/export`. The schema source of truth is `docs/reference/DATASET_MANIFEST.md` (`s4.dataset-manifest.v2`, eligibility policy `s8.2-task-specific-v1`).

9.1 Task-specific readiness

Both tasks need all of the following:

- valid source provenance (a SHA-256 identity);
- queue state INCLUDED;
- admission FUNDUS_ACCEPTED;
- quality neither UNGRADABLE nor NEEDS_REVIEW;
- a confirmed image (`reviewed_by` and `reviewed_at`, or a legacy-compatible record).

On top of that, each task has one extra requirement:

Table 9.1: Readiness rules.

Task	Extra requirement	Reason code when missing
DR-ready (dr_grade_training_ready)	A final grade with source AI_ACCEPTED / AI_CORRECTED / MANUAL, plus reviewer and timestamp	NO_FINAL_CLINICIAN_GRADE, MISSING_PROVENANCE
Lesion-ready (lesion_training_ready)	A current Confirm Annotation whose hash equals the active-set hash, plus reviewer and timestamp	NO_ANNOTATION_CONFIRMATION

AI-only evidence is never promoted to ground truth. Untouched AI, human, corrected, removed, and CVAT-imported annotations stay distinguishable in annotations.csv (annotation_source, original_label, original_score, original_geometry_json).

9.2 Export package

export() writes <output>/dataset-export-<UTC timestamp>-<8 hex>/. The directory is created exclusively, and it is removed again on any failure. It contains:

- manifest.json: counts, coordinate_system: original_image_pixels, lesion taxonomy, grouping_policy: patient_key_only; no automatic train-validation-test split, and the SHA-256 of both CSV files;
- images.csv: one row per image, with the fields listed in IMAGE_FIELDS;
- annotations.csv: one row per annotation, with the fields listed in ANNOTATION_FIELDS.

Source images and case records are never modified by an export. training_group_key (patient:<key>) is exported so that any future experiment can split by patient. The software itself creates no split.

10 API and Contract Map

All request bodies are Pydantic Contract models: extra="forbid", with explicit bounds. Mutations use optimistic revision. Frontend types mirror the responses in frontend/src/lib/api.ts.

10.1 Review workstation API

Table 10.1: Review API.

Method and path	Purpose	Contract / handler
GET /health	Liveness and device snapshot	_factory.health
GET /v1/models	Provider descriptors, degraded per provider rather than failing	_factory.models
GET / PUT /v1/model-connection, POST /v1/model-connection/test	Read, test, or save the remote Model API connection (saves only after a verified probe)	ModelConnectionInput / ModelConnectionResponse; services/model_gateway.py
GET /v1/workspaces, GET /v1/workspaces/active	Catalog and active profile	api/workspaces.py
POST /v1/workspaces, PUT / DELETE /v1/workspaces/{id}, POST .../{id}/open	Create, update, delete (catalog only), open	contracts/workspaces.py

Method and path	Purpose	Contract / handler
POST /v1/workspaces/pickers/{folder,database}	Native pickers	services/pickers.py
POST /v1/admissions/scan	Scan the active input folder	scan_active_workspace
GET /v1/cases, GET /v1/cases/{id}	Worklist rows and case detail (with admission_ui, resolver_ui, review_evidence)	workflow.detail
GET /v1/images/{id}, .../display	Source bytes; browser-safe derivative with lineage headers	workflow.image_bytes, display_image_bytes
POST /v1/cases/{id}/confirm-image	Confirm Image milestone	ConfirmImage
POST /v1/cases/{id}/resolver	Separate patient/eye action	ResolverReview
POST /v1/cases/{id}/admission	Admission override	AdmissionReview
POST /v1/cases/{id}/queue	Exclude or restore	QueueAction
POST /v1/infer/global, /v1/infer/lesion-roi	Optional inference	InferenceRequest → GlobalResult / LesionResult
POST /v1/cases/{id}/review	Confirm DR Grade, escalate, Confirm Annotation	Review
PUT /v1/cases/{id}/annotations	Save human annotation set	HumanAnnotationSave
POST /v1/cases/{id}/lesion-review	AI ROI correction or removal	LesionReviewAction
POST /v1/cases/{id}/cvat/{send, sync}, .../manual-sync	Optional CVAT round trip	Sync, ManualImport
GET /v1/cases/{id}/export	Case detail export	workflow.export
GET /v1/dataset/manifest, POST /v1/dataset/export	Dataset preview and export	services/dataset.py
/app/*, /ui/*	React SPA with fallback to index.html; legacy UI	_factory static mounts

10.2 Model API (GPU host)

Table 10.2: Model API.

Method and path	Auth	Purpose
GET /health	None	Runtime, device, and asset status
GET /v1/models	None	Descriptors for retfound-aptos5 and prism-dr-5fold
POST /v1/predict/dr	Bearer when REMOTE_MODEL_TOKEN is set	RemotePredictRequest → GlobalResult
POST /v1/predict/lesions	Bearer when REMOTE_MODEL_TOKEN is set	RemotePredictRequest → LesionResult

The review/Model API split and the result schemas are **protected contracts**. Change them only through an owner-approved specification, and update providers/remote.py, services/model_api.py, tests/test_remote.py, and tests/test_profiles.py together.

11 Testing and Validation

11.1 Commands

Run from the repository root. On Windows, use `npm.cmd` when PowerShell blocks `npm.ps1`.

```
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m ruff check dr_support tests
cd frontend && npm test && npm run typecheck && npm run build && cd ..
npm test
python scripts/docs/check_docs.py
git diff --check
```

11.2 What the suites cover

Table 11.1: Test suites.

Suite	Files	Focus
Backend pytest	tests/test_admission.py, test_integrity.py, test_dicom_ingest.py, test_derivatives.py, test_imaging_foundation.py test_resolver.py, test_workflow.py, test_dataset.py, test_workspaces.py, test_sync.py, test_bridge.py test_profiles.py, test_remote.py, test_device.py, test_startup.py, test_s5_integration.py, test_pre_s3_hardening.py, test_demo_admission.py	Admission rules, SHA-256 integrity, duplicates and source changes, DICOM scope, derivatives and coordinate mapping Resolver, review actions and concurrency, readiness and export, Workspace catalog, CVAT sync Profile invariants, remote contract, device strictness, integration
Frontend Vitest	frontend/tests/*.test.ts(x)	Shell, Workspace manager, Worklist, admission and resolver dialogs, review, retinal canvas, annotation editor, lesion correction, reviewer preference, datasets
Root Node smoke	tests/overlay.test.cjs, api_hardening.test.cjs, ui.test.cjs	Legacy overlay renderer, API error handling, legacy UI against a live <code>dr_support.api:app</code>
Documentation	scripts/docs/check_docs.py	Local links, requirement IDs, presentation reveal and script synchronisation, stale references

11.3 Known environment prerequisites

- `tests/ui.test.cjs` starts `.venv/Scripts/python.exe` (a Windows path) and expects **11 cases**: the synthetic fixture plus the ten public HRF samples. Fetch the samples first with `python -m dr_support.fetch_samples` (needs internet access to Hugging Face). Without them the test times out waiting for 11 rows. This is an environment prerequisite, not a code defect.
- The frontend build warns about a chunk larger than 500 kB. This is expected for the current bundle.
- Browser-level pixel checks are not part of CI. `jsdom` validates DOM behaviour only.

12 Deployment, Security, and Privacy

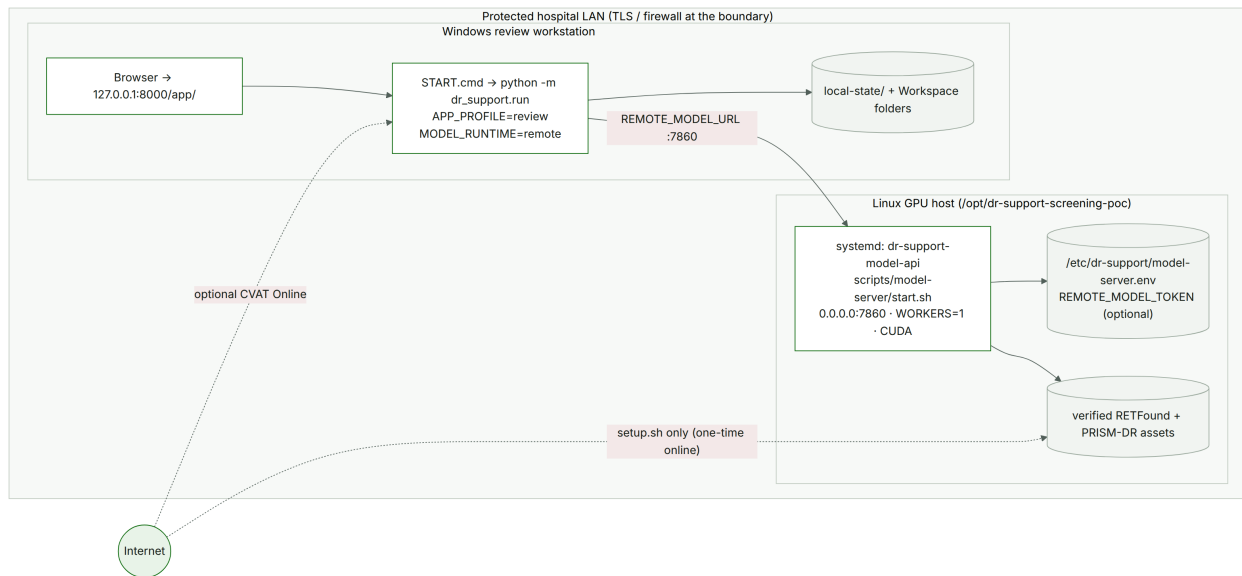


Figure 12.1: Deployment topology. Source: docs/adr/architecture/07-deployment-topology.mmd.

12.1 Topology

- The **workstation** (Windows) runs `START.cmd` and binds to `127.0.0.1:8000`, so the review API is local to that machine. Its state lives in `local-state/` and in the `Workspace` folders.
- The **GPU host** (Linux) runs the `systemd` unit `dr-support-model-api`, which executes `start.sh` on `0.0.0.0:7860` with one worker. It must sit on the protected hospital LAN. TLS, authentication at the boundary, and firewall rules are hospital infrastructure. The optional Bearer token lives in `/etc/dr-support/model-server.env`.
- **Internet** is needed only for `setup.sh` and, optionally, for CVAT Online. Offline acceptance covers `/health`, `/v1/models`, and local inference.

Operational procedures (install, upgrade, backup, restore, troubleshooting) are in the *Deployment and Operations Manual*. They are not repeated here.

12.2 Security and privacy controls in code

Table 12.1: Security and privacy controls.

Control	Implementation
No weights on the workstation	<code>APP_PROFILE=review</code> requires <code>MODEL_RUNTIME=remote</code> (<code>dr_support/app.py</code>).
Fail-closed assets	<code>MODEL_REQUIRE_VERIFIED_ASSETS=1</code> ; <code>setup_models --verify</code> checks revisions and SHA-256 before serving.
Secrets server-side only	<code>REMOTE_MODEL_TOKEN</code> and <code>CVAT_TOKEN</code> are read from the environment. The token is redacted and never returned; there is no browser storage of secrets.
Constant-time token check	<code>_ct_eq</code> in <code>services/model_api.py</code> .
Same-origin mutations	HTTP middleware in <code>api/_factory.py</code> .
No absolute paths in audit or export	Admission <code>source_reference</code> is <code>WORKSPACE_INPUT/<name></code> .

Control	Implementation
Pseudonymous patient keys	normalize_patient_key pattern; the export exposes only a pseudonymous key.
Immutable sources	No code path writes to the input folder. Profile deletion removes the catalog entry only.
Bounded payloads	20 MB base64 limit for analysis images; at most 500 annotations per save.

Never commit PHI, credentials, model weights, runtime databases, local-state/, or render outputs. .gitignore covers local-state/, .env*, *.pt, *.pth, *.zip, node_modules/, and dist/. See docs/operations/SECURITY_PRIVACY.md for incident handling.

13 Documentation as Code

All documentation lives in this repository. It is reviewed like code and built by one script, scripts/docs/build_docs.py.

Table 13.1: Documentation outputs.

Build target	Document	Output
clinician-manual	Clinician User Manual	docs/pdfs/CLINICIAN_USER_MANUAL.pdf
operator-manual	Deployment and Operations Manual	docs/pdfs/DEPLOYMENT_OPERATIONS_MANUAL.pdf
sampling	Team Image Sampling Requirements	docs/pdfs/IMAGE_SAMPLING_REQUIREMENTS.pdf
developer	Developer Technical Guide	docs/pdfs/DEVELOPER_TECHNICAL_GUIDE.pdf
image-guide	Clinician Image Selection Guide	docs/pdfs/CLINICIAN_IMAGE_SELECTION_GUIDE.pdf
demo	Product demo	docs/presentation/RETINAL_REVIEW_DEMO.html
briefing	Technical briefing	docs/presentation/TECHNICAL_BRIEFING.html
diagrams	Architecture diagrams	docs/adr/architecture/rendered/*.png, *.svg

Sources: docs/manuals/clinician/, docs/manuals/operator/, docs/clinician/sampling/ (with REQUIREMENTS_TRACEABILITY.md), docs/manuals/developer/ (Quarto books); docs/clinician/IMAGE_SELECTION_GUIDE.pdf; docs/presentation/hyperframes/index.html (HyperFrames slideshow source); docs/adr/architecture/*.mmd.

```
python scripts/docs/build_docs.py all          # everything
python scripts/docs/build_docs.py demo briefing
python scripts/docs/build_docs.py developer --html # also HTML site under dist/manual/
python scripts/docs/check_docs.py              # links, IDs, reveal/script sync
```

Tools.

- **Quarto** (1.6 or later) with **XeLaTeX** builds the books. Set QUARTO_BIN if Quarto is not on PATH. The script runs Quarto inside each book folder, because Quarto reads a .env.example in the working directory as a list of required variables.
- **Mermaid CLI** (mmdc) renders the diagrams. Set PUPPETEER_EXECUTABLE_PATH to reuse an installed Chromium.
- **Playwright** prints the image guide. Set PLAYWRIGHT_CHROMIUM_EXECUTABLE to reuse an installed Chromium.
- The demo and briefing builds use only the Python standard library. They inline CSS, JS, and images into one offline HTML file.

Rules.

- Keep one source of truth per topic (see `README.md`, *Documentation*).
- After a rebuild, look at every PDF page and every presentation scene, not just the build log.
- A presentation reveal change must be mirrored in the Thai script. `check_docs.py` fails when the counts differ.
- Diagrams use flowchart and sequence syntax with C4 semantics in the labels, because Mermaid's native C4 syntax is still marked experimental.

14 Limitations and Glossary

14.1 Known limitations

- **Identity.** Single-user localhost runtime. The reviewer name is self-declared (`LOCAL_POC_SELF_DECLARED`). A reverse proxy with authentication is needed before any remote hosting.
- **Source class and sex.** The Workbench does not store the source Refer/Not-refer class or sex. Coverage monitoring depends on a team-held record (Section 3).
- **Near-duplicates.** Near-duplicates (resized, re-exported, or repeat captures) are not detected. Only byte-identical files are.
- **Quality checks.** Automatic admission checks are coarse screens, not a gradability model.
- **Resolver.** The resolver uses anchored filename patterns. OCR is off by default. DICOM laterality is extracted but not used by the resolver.
- **Admission scan.** It is top-level only. Files in sub-folders are ignored.
- **Model scores.** They are uncalibrated. An empty PRISM-DR result does not prove absence.
- **Dataset splits.** No automatic train/validation/test split exists. Future experiments must split by `training_group_key`.
- **Root UI smoke test.** It needs the public HRF samples and a Windows-style `.venv/Scripts/python.exe`.
- **Screenshots.** The manual screenshots are committed assets. The capture tooling was removed from main in `ec74af8`, so re-capturing requires restoring or rewriting a capture script against the current UI.

14.2 Glossary

Table 14.1: Glossary.

Term	Meaning
Admission	Conservative decision on whether a staged file is a supported, reviewable retinal image.
Analysis derivative	Deterministic representation sent to a model, with its own SHA-256 and coordinate mapping.
Annotation-set hash	Hash of the active human, AI-decision, and imported annotations. Confirm Annotation binds to it.
Confirmation milestone	Confirm Image, Confirm DR Grade, or Confirm Annotation.
DR-ready / Lesion-ready	Task-specific dataset eligibility.
Model API	GPU-host FastAPI surface (<code>model_api</code> profile).
Profile	<code>APP_PROFILE</code> value: <code>review</code> , <code>model_api</code> , or <code>full</code> .
Resolver	Filename/OCR evidence for the patient key and eye. It is never authoritative on its own.
Source class	The hospital's Refer / Not-refer classification. It is outside the Workbench.

Term	Meaning
Source SHA-256	Hash of the original admitted bytes; the case identity.
Workspace	Profile of input folder, output folder, and review database.